EDUCATIONAL EVENT MANAGEMENT USING SPRING MVC

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

A . Dhanunjaya (VTU25920)

10211CS224
Full Stack Application Development
Slot : E1

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May , 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”EDUCATIONAL EVENT MANAGEMENT USING SPRING MVC” by A . Dhanunjaya (VTU25920) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr . S . Hemamalini

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May , 2026

Signature of Course Co-ordinator

Dr. Sumithra . M.

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May , 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(A . Dhanunjaya)

Date:06/01/2026

ABSTRACT

Educational Event Management Using Spring MVC is a web-based application developed to automate and streamline the organization of academic events such as seminars, workshops, conferences, and campus programs within educational institutions. The system provides a centralized platform where administrators can create, update, and manage events efficiently, while students can securely register, browse upcoming events, and track their participation through a personalized dashboard.

The application is developed using Java 17 and Spring Boot following the Spring MVC layered architecture, which consists of Controller, Service, Repository, and Model components. Spring Security is integrated to implement role-based authentication and authorization for ADMIN and STUDENT users, ensuring secure access control. The system also includes event registration management, validation mechanisms, and basic statistical reporting for administrative monitoring.

MySQL is used as the backend relational database to ensure reliable and structured data storage. Thymeleaf is utilized for dynamic server-side rendering of web pages, providing a responsive and user-friendly interface. The proposed system reduces manual effort, improves data

accuracy, enhances administrative efficiency, and increases student engagement through a secure and scalable event management solution.

Keywords: Educational Event Management, Spring MVC, Spring Boot, Role-Based Access Control, MySQL, Web Application, Event Registration, Spring Security.

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DTO	Data Transfer Object
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JPA	Java Persistence API
JDBC	Java Database Connectivity
JVM	Java Virtual Machine
MVC	Model View Controller
ORM	Object Relational Mapping
RBAC	Role-Based Access Control

RDBMS	Relational Database Management System
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

Educational institutions regularly conduct seminars, workshops, technical symposiums, cultural programs, conferences, and training sessions to enhance student knowledge, skills, and engagement. Traditionally, managing such events involves manual processes such as paper-based registrations, spreadsheet tracking, email communication, and physical coordination. These conventional approaches often result in data redundancy, inefficiencies, lack of transparency, and increased administrative workload.

The *Educational Event Management Using Spring MVC* project is a web-based application developed to automate and streamline the complete lifecycle of campus event management. The system provides a centralized platform where administrators can create, update, delete, and monitor events efficiently. Students can browse available events,

securely register online, and track their participation through a personalized dashboard.

The application is developed using Java 17 and Spring Boot following the Spring MVC architectural pattern. It adopts a layered architecture consisting of Controller, Service, Repository, and Model layers to ensure modularity, scalability, and maintainability. Role-Based Access Control (RBAC) is implemented using Spring Security to provide secure authentication and authorization for ADMIN and STUDENT roles. The system includes event registration management, validation mechanisms, and administrative monitoring features to improve coordination and efficiency. The proposed solution replaces traditional manual event handling with a secure, efficient, and automated digital platform.

1.2 Aim of the Project

The primary aim of this project is to design and develop a secure, scalable, and user-friendly web application for managing educational events using Spring MVC architecture. The system aims to automate event creation, student registration, participation tracking, and administrative monitoring while ensuring secure access control and efficient database management. It also seeks to enhance transparency, reduce

manual workload, and improve overall institutional efficiency.

1.3 Scope of the Project

The scope of this project includes the development of a full-stack web application tailored for managing academic and campus events within an educational institution. The system enables administrators to manage event details, monitor registrations, and analyze participation statistics. Students can securely log in, register for events, and view their registered events through dedicated dashboards.

The project integrates MySQL for structured database management, Spring Security for authentication and authorization, and Thymeleaf for dynamic server-side rendering of web pages. The current implementation focuses on web-based deployment within an institutional environment. Future enhancements may include mobile application integration, advanced analytics dashboards, cloud deployment, and notification mechanisms for improved user engagement.

1.4 Framework

The system is developed using the Spring Boot framework with Spring MVC architecture, which provides structured request handling and separation of concerns. Spring Boot simplifies dependency man-

agement and configuration, enabling efficient development and deployment. Spring Data JPA is used for database interaction and object-relational mapping, while Spring Security ensures secure authentication and role-based authorization.

The frontend is developed using HTML, CSS, JavaScript, and Thymeleaf templates to provide a responsive and interactive user interface. MySQL is used as the relational database for storing user credentials, event details, and registration records. The overall framework ensures modularity, scalability, security, and maintainability of the Educational Event Management System.

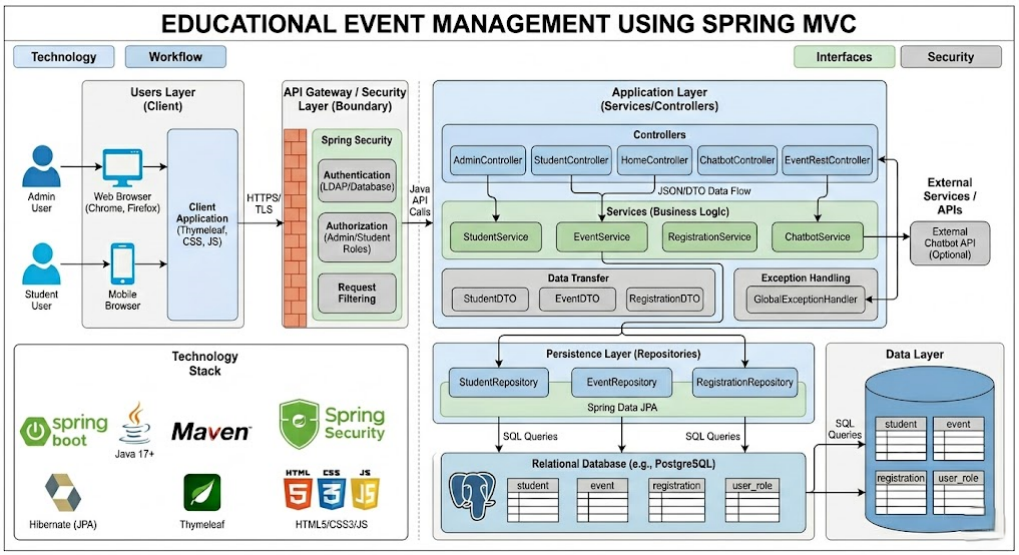

Figure 1.1: System Architecture Diagram

Chapter 2

SURVEY OF SIMILAR APPLICATIONS IN MARKET

The purpose of this survey is to examine existing event management systems available in the market and analyze their features, strengths, and limitations. Understanding these platforms helps identify current industry standards and technological practices. By comparing these solutions with the proposed *Educational Event Management Using Spring MVC* system, research gaps can be identified and the need for a customized academic event management platform can be justified.

2.1 Eventbrite

Eventbrite is a globally recognized online event management and ticketing platform that enables organizers to create, promote, and manage events efficiently. It supports multiple event categories including conferences, workshops, festivals, and professional gatherings. The

platform provides integrated payment processing, attendee tracking, and analytics dashboards.

Despite its comprehensive features, Eventbrite is primarily designed for commercial and revenue-oriented events. It focuses heavily on ticket sales and marketing rather than academic workflows. Premium features require subscription fees, making it less suitable for educational institutions with limited budgets.

- **Pros:** User-friendly interface, analytics support, global accessibility.
- **Cons:** Commercial focus, limited academic customization, subscription cost.

2.2 Google Calendar and Forms Integration

Many institutions use Google Calendar combined with Google Forms to manage event announcements and registrations. Organizers schedule events and share registration links, while responses are stored in Google Sheets for manual processing.

Although this approach is simple and cost-effective, it lacks centralized dashboards, structured database integration, and role-based authentication. Administrative tasks such as tracking registrations and generating reports must be handled manually, increasing workload and

the possibility of data inconsistency.

- **Pros:** Free, easy to implement, widely accessible.
- **Cons:** Manual processing, limited automation, no structured access control.

2.3 Microsoft Teams Events

Microsoft Teams provides webinar and meeting scheduling features within the Microsoft 365 ecosystem. It supports virtual event hosting, invitations, and basic attendance reporting.

However, it is primarily designed for online meetings rather than complete event lifecycle management. It does not provide dedicated administrative dashboards for campus event tracking or structured student registration workflows. Access also depends on institutional subscription plans.

- **Pros:** Secure enterprise environment, strong collaboration features.
- **Cons:** Focused on virtual meetings, limited event management customization.

2.4 Campus Event Management Applications

Some universities deploy dedicated campus event management systems developed in-house or purchased as institutional software. These platforms usually provide event browsing and registration features.

However, many such systems lack scalability, structured role-based access control, and advanced reporting features. Customization options are often limited, and integration with modern development frameworks may be minimal.

- **Pros:** Designed for academic use, focused event browsing.
- **Cons:** Limited scalability, minimal analytics, restricted customization.

2.5 Summary of Survey

The survey indicates that existing solutions either prioritize commercial functionality or provide basic event coordination tools without strong automation and institutional customization. Commercial platforms may offer rich features but are costly and not specifically tailored to academic workflows. Simple integration tools lack automation and secure authentication mechanisms. Some campus systems provide partial solutions without scalable architecture.

The proposed *Educational Event Management Using Spring MVC* system addresses these gaps by providing a secure, scalable, and institution-focused platform. It integrates role-based authentication, centralized dashboards, structured database management, and automated registration handling within a unified framework. This ensures improved efficiency, enhanced security, and better administrative control compared to existing market solutions.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the Educational Event Management system is developed using HTML5, CSS3, JavaScript, and the Thymeleaf template engine. The development process is carried out using Visual Studio Code for interface design and Google Chrome for browser-based testing and debugging. Thymeleaf is integrated with Spring Boot to enable dynamic server-side rendering and seamless data binding between backend controllers and frontend templates.

Static resources such as CSS, JavaScript, and image files are organized under the `resources/static` directory, while HTML template files are stored in the `resources/templates` directory. This structured organization ensures maintainability and separation of presentation logic.

3.1.2 Structure of the Project

The frontend follows a modular and role-based structure. Template files are organized into folders such as `admin`, `student`, `events`, and `error` to maintain clarity and logical grouping. Reusable components including navigation bars and layout sections are implemented to ensure UI consistency across all pages.

Styling is managed through a centralized stylesheet (`style.css`), while dynamic interactions and client-side validations are handled using JavaScript (`main.js`). This structured approach enhances code reusability, readability, and scalability.

3.1.3 Execution Procedure

The frontend runs as part of the Spring Boot application. When the application is started using Maven or directly from the IDE, the embedded Tomcat server runs on the default port 8080. Users can access the system via `http://localhost:8080`.

Based on authentication and assigned roles (ADMIN or STUDENT), Spring Security dynamically grants access to authorized dashboards and pages. Thymeleaf templates receive model data from controllers and render dynamic web content accordingly.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed using Java 17 and the Spring Boot framework. Development is carried out using an Integrated Development Environment (IDE) such as IntelliJ IDEA or Eclipse. Maven is used for build automation and dependency management.

Spring Boot simplifies configuration through auto-configuration, embedded server support, and dependency injection, reducing development complexity and improving maintainability.

3.2.2 Structure of the Project

The backend follows a layered architecture to ensure separation of concerns and maintainability. The primary layers include:

- **Controller Layer** – Handles HTTP requests and returns appropriate views or responses.
- **Service Layer** – Contains core business logic such as event management and registration processing.
- **Repository Layer** – Interacts with the database using Spring Data JPA.
- **Model Layer** – Represents database entities mapped using JPA annotations.

- **DTO Layer** – Transfers structured data between client and server.

Spring Security is implemented to provide authentication and role-based authorization. Business rules such as preventing duplicate registrations are handled within the service layer.

3.2.3 Execution Procedure

The backend application is executed by running the main class (`CampusEve` Maven automatically resolves and downloads required dependencies. Upon execution, the embedded Tomcat server initializes and establishes a connection with the MySQL database using configurations defined in the `application.properties` file.

Once successfully started, the application becomes accessible through the configured server port.

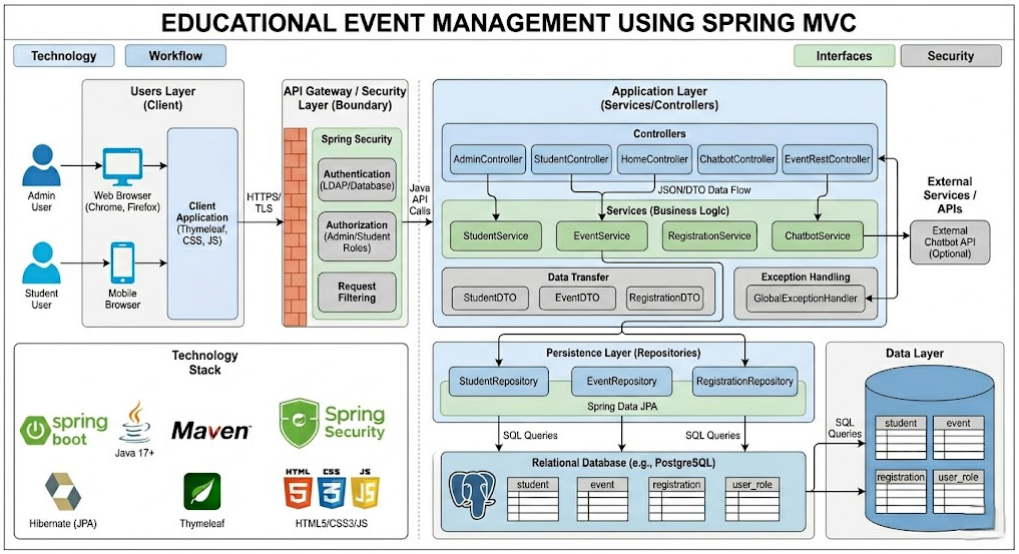

Figure 3.1: System Framework Architecture

3.3 Database Implementation

3.3.1 Database Configuration

The system uses MySQL as the relational database management system for persistent data storage. Database connection parameters such as JDBC URL, username, password, and Hibernate dialect are configured in the `application.properties` file.

Spring Data JPA is used for object-relational mapping, while Hibernate manages schema generation and table creation automatically.

3.3.2 Schema Structure

The database schema consists of core tables including Student, Event, and Registration. Primary and foreign key constraints are defined to maintain referential integrity.

The Registration table acts as a junction table establishing a relationship between students and events. Entity relationships are mapped using JPA annotations such as `@Entity`, `@Id`, `@OneToMany`, `@ManyToOne`, and `@JoinColumn`.

3.3.3 Tables Created

- **Student** – Stores student information including ID, name, email, encrypted password, and assigned role.
- **Event** – Stores event details such as title, description, date, venue, and organizer information.

- **Registration** – Stores registration records linking students to specific events along with registration timestamp.

The database design ensures data integrity, consistency, and secure storage of user credentials and event participation details.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The Educational Event Management System follows a three-tier architecture consisting of the Presentation Layer, Business Logic Layer, and Data Access Layer. This architecture ensures modular development, maintainability, and scalability of the application.

- **Presentation Layer:** Developed using HTML, CSS, JavaScript, and Thymeleaf templates to provide interactive and dynamic user interfaces. It handles user requests and displays appropriate responses.
- **Business Logic Layer:** Implemented using Spring Boot service classes that process application logic such as event creation, registration validation, and access control.
- **Data Access Layer:** Uses Spring Data JPA and Hibernate ORM

framework to interact with the MySQL database and manage persistent data storage.

The system follows the Spring MVC design pattern, separating components into Controller, Service, and Repository layers. This separation of concerns improves code organization, security, and extensibility.

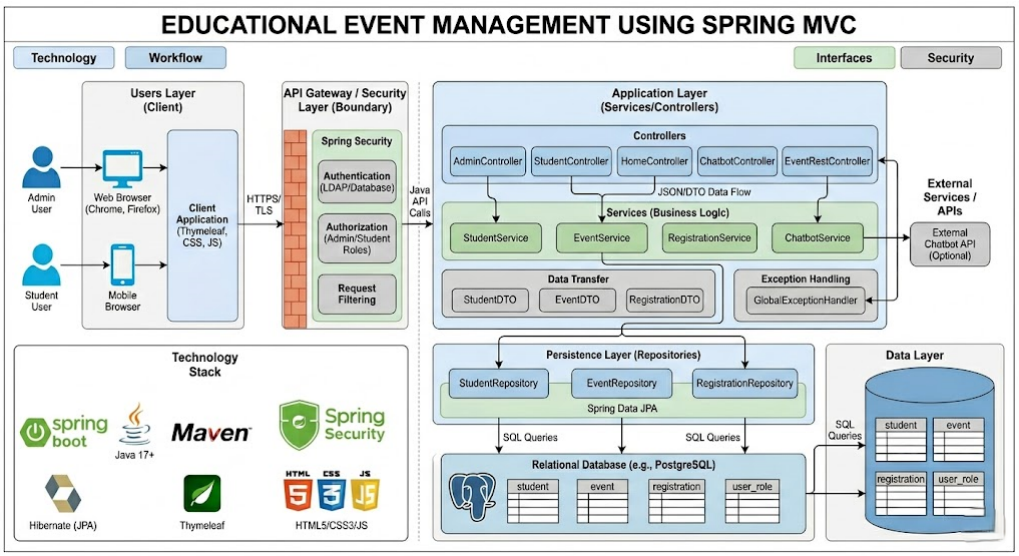

Figure 4.1: System Architecture Diagram

4.2 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates how data flows within the system and how different entities interact with various processes.

- Students register and log in using secure authentication.
- Students browse available events and submit registration requests.

- The system validates and stores registration data in the database.
- Administrators create, update, and manage event details.
- Registration records are maintained for monitoring and reporting.

The DFD represents interactions between external entities (Student and Admin), internal processes (Authentication, Event Management, Registration Processing), and the central database.

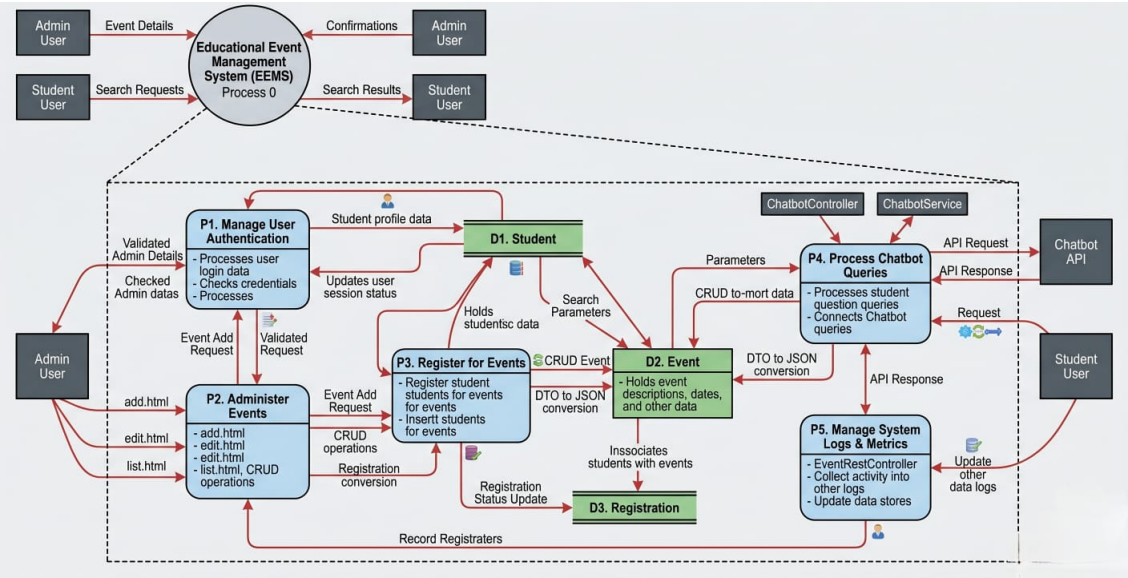

Figure 4.2: Data Flow Diagram

4.3 Module 1: User Management Module

The User Management Module handles authentication, authorization, and user account management using Spring Security.

- Implements secure login and logout mechanisms.

- Supports Role-Based Access Control (RBAC) for ADMIN and STUDENT roles.
- Encrypts passwords using secure hashing algorithms (e.g., BCrypt).
- Manages session control and request authorization filters.
- Restricts access to pages based on assigned roles.
- Provides input validation and secure credential handling.

This module ensures that only authorized users can access system functionalities.

4.4 Module 2: Event Management Module

The Event Management Module manages the complete lifecycle of events within the institution.

- Allows administrators to create events with title, description, date, and venue.
- Supports updating and deleting existing events.
- Displays lists of upcoming and past events.
- Enables students to browse and view event details.
- Maintains participation statistics for administrative monitoring.

This module centralizes event operations and ensures structured event handling.

4.5 Module 3: Registration Module

The Registration Module manages student participation records.

- Processes student event registration requests.
- Stores registration records securely in the database.
- Maintains mapping between students and events.
- Prevents duplicate registration entries through validation logic.
- Displays registered events in the student dashboard.

This module ensures accurate tracking of event participation.

Advantages of this Project

- **Improved Efficiency:** Automates event management and reduces manual workload.
- **Enhanced Security:** Implements Spring Security with encrypted passwords and role-based access control.
- **Centralized Data Management:** Maintains structured and consistent data storage using MySQL.

- **User-Friendly Interface:** Provides intuitive dashboards for students and administrators.
- **Scalability:** Can be extended with additional modules and deployed in cloud environments.
- **Paperless Operation:** Reduces paperwork through online registration.

Disadvantages

- Requires stable internet connectivity.
- Initial deployment and configuration require technical knowledge.
- System performance may depend on server capacity during high traffic.
- Regular maintenance and updates are necessary.

Chapter 5

RESULTS AND DISCUSSIONS

The Educational Event Management System was successfully developed and tested using Spring Boot, Thymeleaf, and MySQL. The system was evaluated based on functionality, usability, performance, and security aspects.

The following results were observed:

- Students were able to register and login securely using authenticated credentials.
- Admin users successfully performed Create, Read, Update, and Delete (CRUD) operations on events.
- The system prevented duplicate event registrations through validation logic.
- Role-Based Access Control (RBAC) was properly enforced using Spring Security.

- Event data and registration records were stored and retrieved efficiently from the MySQL database.
- The dashboards dynamically displayed event and registration information.

The system was tested with multiple user accounts to ensure reliability and consistent performance. All core modules functioned as expected without data loss or security issues.

Performance Comparison

Table 5.1: System Comparison with Existing Methods

Application Type	**Efficiency Level**
Manual Event Management	Low
Spreadsheet-Based System	Moderate
Basic Web Portal	Moderate
Educational Event Management (Proposed System)	High

The comparison shows that the proposed system provides higher efficiency due to automation, centralized database management, and secure role-based access control.

System Screenshots

The following screenshots demonstrate the working of the system including login page, home page, student dashboard, admin dashboard,

event page, statistics page, and event registration page.

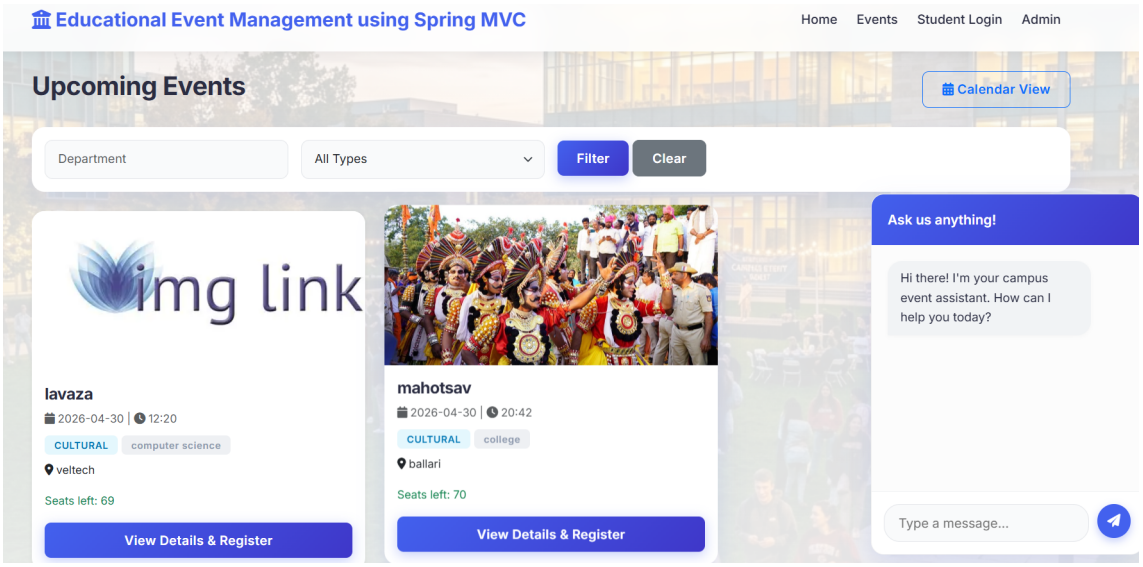

Figure 5.1: Home Page

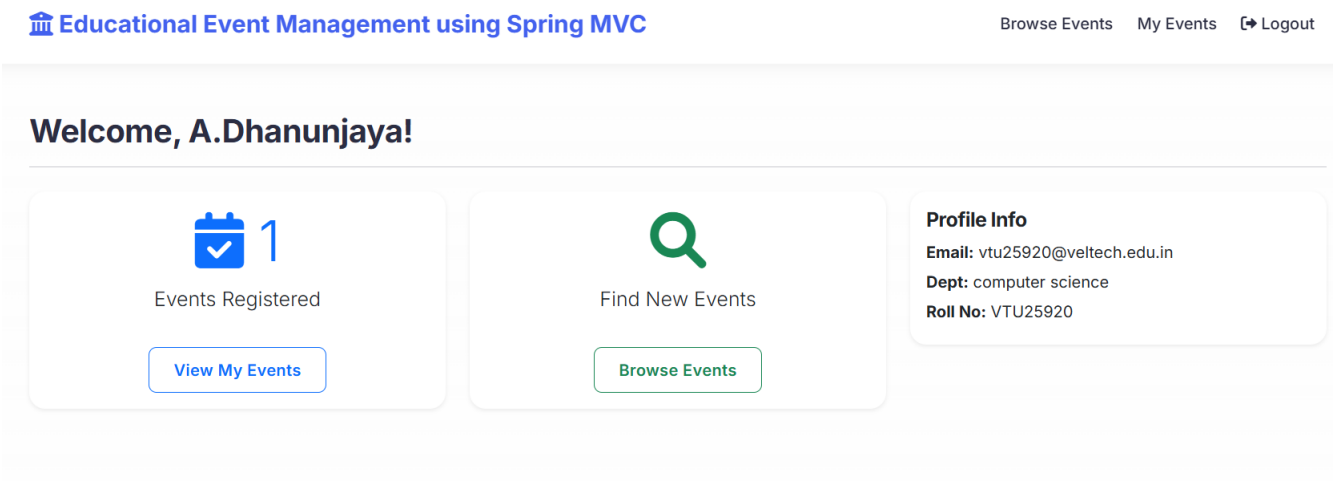

Figure 5.2: Student Dashboard

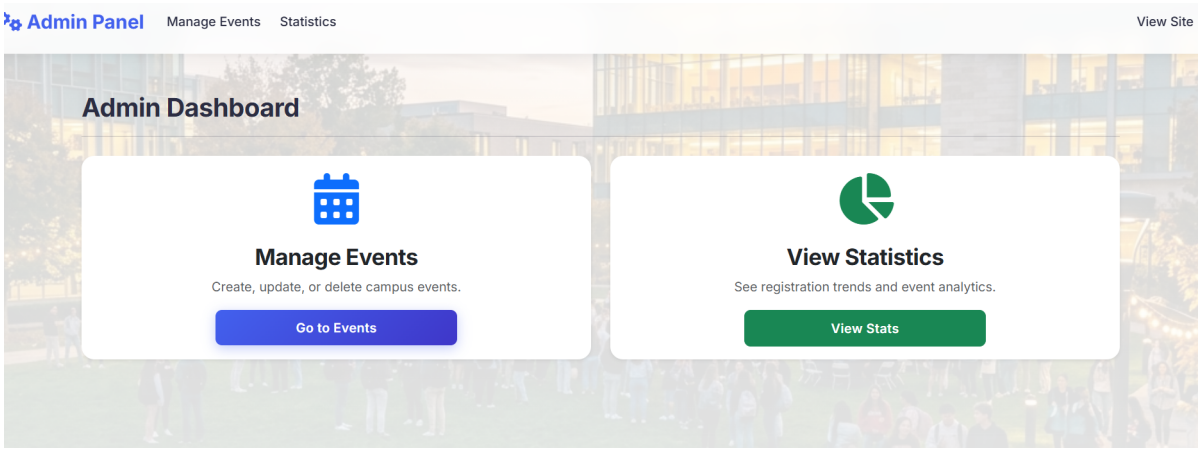

Figure 5.3: Admin Dashboard

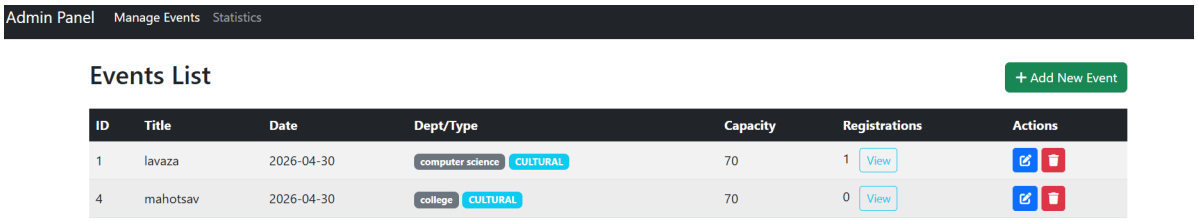

Figure 5.4: Event Listing Page

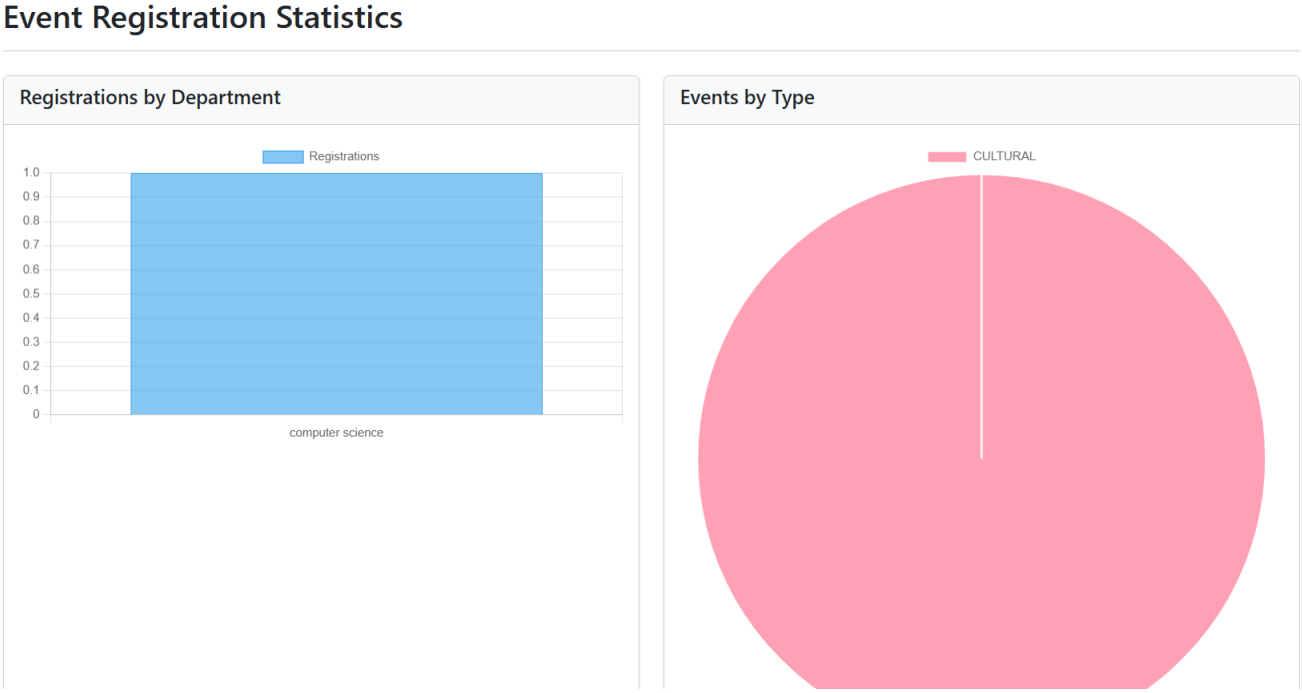

Figure 5.5: Event Statistics Page

The results clearly indicate that the proposed system improves automation, enhances data accuracy, reduces administrative workload, and provides a better user experience compared to traditional event management methods.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Educational Event Management System developed using Spring Boot and Spring MVC provides a secure, structured, and efficient platform for managing academic events such as seminars, workshops, symposiums, and conferences within educational institutions.

The system successfully automates key processes including event creation, student registration, participation tracking, and administrative monitoring. By implementing Role-Based Access Control (RBAC), the application ensures that administrators and students access only authorized functionalities. Secure authentication using Spring Security and structured data management through MySQL enhance reliability and data integrity.

The layered MVC architecture improves modularity, maintainabil-

ity, and scalability of the system. Overall, the project reduces manual workload, increases operational efficiency, enhances transparency, and delivers a centralized digital solution for institutional event management.

6.2 Future Enhancements

The system can be further enhanced to improve functionality and scalability. Possible future improvements include:

- Implementation of email and SMS notification systems for automated event reminders.
- QR code-based attendance marking for physical event verification.
- Integration of an online payment gateway for managing paid events.
- Development of a mobile application using REST APIs for wider accessibility.
- Advanced analytics dashboard for detailed participation and performance insights.
- Deployment on cloud platforms for improved scalability and availability.

These enhancements would further strengthen the system and expand its applicability across larger educational institutions.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	In-depth engineering knowledge	WK3, WK4, WK5, WK6	Applies Spring Boot, Spring MVC, database design, and security concepts to develop a scalable academic event management system.
WP2	Conflicting Requirements	Technical and usability conflicts	WK4, WK5, WK6	Balances secure authentication and user-friendly interface while maintaining data validation and access control.
WP3	Depth of Analysis	Analytical system design	WK3, WK4, WK5	Involves system architecture design, schema mapping, validation logic, and role-based authorization analysis.
WP4	Familiarity	New institutional problem	WK4, WK8	Develops a centralized digital solution replacing manual and spreadsheet-based event handling systems.
WP5	Applicable Codes	Custom-built solution	WK6	Implements CRUD operations, registration logic, duplicate prevention, and secure login mechanisms.
WP6	Stakeholder Needs	Multiple user roles	WK5, WK6, WK7	Supports Admin and Student roles with controlled access using Spring Security.
WP7	Interdependence	Integration of subsystems	WK3, WK5, WK6	Integrates frontend (Thymeleaf), backend (Spring Boot), and MySQL database into one unified system.

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification Based on the Project
EA1	Range of Resources	Use of diverse technologies	Utilizes Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML, CSS, JavaScript, and MySQL for complete full-stack implementation.
EA2	Level of Interactions	Complex module interactions	Manages interactions between authentication, event management, registration processing, and database modules.
EA3	Innovation	Application of modern technologies	Provides a centralized web-based academic event management platform with role-based security and automated validation.
EA4	Consequences to Society	Institutional impact	Enhances transparency, improves student participation, and reduces administrative workload and manual errors.
EA5	Familiarity	Beyond traditional practices	Replaces manual systems with secure digital automation and centralized monitoring dashboard.

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Facilitates student participation in academic events, workshops, and seminars, improving learning opportunities beyond classroom education.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in educational institutions through a secure web-based management system.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives by digitizing event processes and improving institutional coordination.
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling online event registration and management.
SDG 16	Peace, Justice and Strong Institutions	Ensures transparency and secure access control through authentication and role-based authorization.

Table 7.3: SDG Mapping for the Project

References

[1] Spring Boot Documentation.
https://spring.io/projects/spring-boot

[2] Spring Security Documentation.
https://spring.io/projects/spring-security

[3] Thymeleaf Official Documentation.
https://www.thymeleaf.org

[4] MySQL Documentation.
https://dev.mysql.com/doc

[5] Hibernate ORM Documentation.
https://hibernate.org/orm/documentation/